Day 5 – Orders API: Buying a Game Key

Goals: Build the purchase flow — atomically assign a key and set its expiry.

Add `Order` and `OrderItem` to `games/models.py`:

```
class Order(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    purchased_at = models.DateTimeField(auto_now_add=True)


class OrderItem(models.Model):
    order = models.ForeignKey(Order, on_delete=models.CASCADE)
    game_key = models.OneToOneField(GameKey, on_delete=models.CASCADE)
```

`games/views.py` — `POST /api/orders/`:

```
import uuid
from datetime import timedelta
from django.utils import timezone
from django.db import transaction
from rest_framework.decorators import api_view, permission_classes
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response
from rest_framework import status
from .models import Game, GameKey, Order, OrderItem


@api_view(['POST'])
@permission_classes([IsAuthenticated])
def create_order(request):
    game_id = request.data.get('game_id')
    if not game_id:
        return Response({'error': 'game_id is required.'}, status=statu

    try:
        game = Game.objects.get(id=game_id)
    except Game.DoesNotExist:
        return Response({'error': 'Game not found.'}, status=status.HTT

    with transaction.atomic():
        # Use select_for_update to prevent race conditions
        existing_key = (
            GameKey.objects.select_for_update()
            .filter(game=game, status='active', owner__isnull=True)
            .first()
        )

        if existing_key:
            # Assign existing unowned key
            key = existing_key
```

```
            key.owner = request.user
            key.save()
        else:
            # Generate a fresh key
            key = GameKey.objects.create(
                key_string=str(uuid.uuid4()).upper(),
                game=game,
                status='active',
                expires_at=timezone.now() + timedelta(days=30),
                owner=request.user,
            )

        order = Order.objects.create(user=request.user)
        OrderItem.objects.create(order=order, game_key=key)

    return Response({
        'order_id': order.id,
        'game': game.title,
        'key': key.key_string,
        'expires_at': key.expires_at,
    }, status=status.HTTP_201_CREATED)
```

Wire URL: `path('api/orders/', create_order)` in `urls.py`.

🎓 Instructor Teaching Plan

⏱ Time Breakdown (90 min)

Segment	Duration	Activity
Race condition motivation	20 min	Before writing any code, pose the problem: two users hit POST /api/orders/ at the exact same millisecond for the same game. Without locks, they both read the same available key and both get assigned it — a key is sold twice. Draw the timeline on a whiteboard.
Add Order & OrderItem models	15 min	Add models, run `makemigrations` + `migrate`. Register in admin.
Walk through `create_order` view	30 min	Go line by line. Emphasize `transaction.atomic()`, `select_for_update()`, the `owner__isnull=True` filter. Explain why we generate a fresh key if no pre-loaded keys exist.
Wire URL & live test	15 min	Add URL, test with Postman. Create a game through admin, then buy it. Show the key and expiry in the response.
Q&A + checkpoint	10 min	

💡 Key Concepts to Introduce

- **`transaction.atomic()`** — everything inside the `with` block executes as one DB transaction. If any exception is raised, all changes are rolled back.
- **`select_for_update()`** — places a row-level lock on the selected rows. Any other

transaction trying to lock the same rows will wait until this transaction completes. This is *pessimistic locking*.
- **Race condition** — a bug that depends on the timing of concurrent operations. Hard to reproduce in development (single user), catastrophic in production (many users).
- **`auto_now_add=True`** on `Order.purchased_at` — automatically sets to `timezone.now()` on creation. Read-only after creation. Compare with `auto_now=True` (updates on every save).
- **`uuid4()`** for key generation — cryptographically random, collision probability negligible. Better than auto-increment integers for keys that users will see.
- **`timedelta(days=30)`** — adding a duration to a datetime. Explain `timezone.now()` vs `datetime.now()` — always use `timezone.now()` in Django to stay timezone-aware.

⚠ Common Mistakes to Address

- **Not using `transaction.atomic()`** — the key is locked but if an exception happens after the lock, the key status is left in an inconsistent state.
- **Filtering with `status='active'` but not `owner__isnull=True`** — would return keys that are already owned by someone else.
- **Not capturing keys before `update()`** — in Day 6/8, students will update the queryset with `.update(status='expired')`, which clears the queryset. Foreshadow: always materialize (`list()`) before bulk-updating.
- **Returning the raw `key_string` field vs a formatted version** — fine for now, but flag that in production you'd never expose this in a GET endpoint (only return once, at purchase time).
- **Missing `@permission_classes([IsAuthenticated])`** — anyone could buy keys without registering.

? Check for Understanding

> "What would go wrong without `select_for_update()`?"
>
> **Answer:** Without `select_for_update()`, a race condition could occur if two concurrent requests query the database at the same time for an available key. Both would find the same unowned key, assign it to their respective user, and write it back. This results in the same game key being sold twice (double-allocation).
>
> "When does `transaction.atomic()` roll back? What triggers a rollback?"
>
> **Answer:** It rolls back when an unhandled exception is raised within the context manager block. If any error (like a database constraint violation or a Python runtime exception) occurs before the block exits successfully, all changes made to the database within that block are discarded.
>
> "Why do we filter with `owner__isnull=True`? What does `owner` represent for a key that no one has purchased yet?"
>
> **Answer:** We filter for `owner__isnull=True` to retrieve a key that has not yet been bought by any user. For an unpurchased key, `owner` is `None` (represented as `NULL` in the database). Filtering this way prevents re-allocating an already purchased key.
>
> "Why use `uuid4()` to generate the key string instead of, say,

`GameKey.objects.count() + 1`?"

Answer: `uuid4()` generates cryptographically random, unique identifiers that cannot be guessed by users, preventing attackers from predicting and stealing other game keys. Using `count() + 1` is sequential, exposing the total volume of keys and making them trivial to guess and exploit.

✓ Day Checkpoint

`POST /api/orders/` with a valid `game_id` and auth token returns `201` with `order_id`, `game`, `key`, and `expires_at`. `POST /api/orders/` without a token returns `401`. `POST /api/orders/` with a non-existent `game_id` returns `404`.